

JAVA

Scenario: Ram and Shyam opened acc in PNB. The bank opens the acc with initial balance of 5k.

Ram deposited 20000 more and Shyam deposits 4000.

Shyam lends 5000 to Ram.

- Create reference var Ram and Shyam
- initial bal shld be 5k(compul)

new is object creation

Bank b=new bank();

()---> is constructor;

Constructor is a piece of code which is executed right after obj creation .JVM invokes constructor.

```
class BankAccount
{
    String accHolderName;
    float balance;
    String bankName;
    Bank Account(String name,String bN)
    {
        balance=5000;
        //bankName="PNB";
        bankName=bN;
        accHolderName=name;
        System.out.println("Account Created");
    }
    Bank Account(String name)
    {
        balance=5000;
        bankName="PNB";
        //bankName=bN;
        accHolderName=name;
        System.out.println("Account Created");
    }
    void deposit(float balance)
    {
        this.balance=this.balance+balance;
    }
    void withdraw(float amount)
    {
        if (amount > (balance-1000))
        {
```

```

        System.out.println("Insufficient Funds");
        return;
    }
    balance-=amount;
}
float checkBalance()
{
    return balance;
}
}

```

CODE::

```

class Bank
{
    public static void main(String[] args)
    {
        Bank Account ram=new BankAccount("Ram");
        Bank Account shyam=new BankAccount("Shyam","PNB");
        Bank Account radhika=new BankAccount("Radhika","Canara Bank");
        //Bank Account priya=new BankAccount();//error coz there is no
default constructor in the prog
        System.out.println(ram.accHolderName+ " has " + ram.balance+ " in
"+ ram.bankName);
        ram.deposit=10000;
        System.out.println(ram.balance);// wont change as you are changing
the balance only in deposit and JVM wont take instance variable ka balance it
will print parameter wala balance and for this not to happen and for the jvm to
take instance variable ka balance we us "this" keyword( when parameter and
instance variable is same)
        System.out.println(ram.accHolderName+ " has " + ram.balance+ " in
"+ ram.bankName);
        System.out.println(shyam.accHolderName+ " has " + shyam.balance+
" in "+ shyam.bankName);
        System.out.println(radhika.accHolderName+ " has " +
radhika.balance+ " in "+ radhika.bankName);

    }
}

```

this:

this is a reference variable pointing to current object.

shadowing:

when the input parameter shares the same name as that of instance variable, the shadow of the input parameter falls on the instance variable and the

instance variable remains unchanged. This concept is called shadowing. Due to shadowing the JVM is unable to distinguish between the parameter and the instance variable. To get rid of shadowing we use this keyword.

if the constructor name and the parameters are same then there will be error but if the parameters are changed then there will be no error

are objects are capable of having multiple states at the time of creation??

Yessss how? constructors types.

there are 2 types of constructors

implicit and explicit and default

implicit: when u have only one constructor which is parameterized

default constructor is given when you don't initialise any constructor

CONSTRUCTOR

code block used to initialise objects

has same name as the class in which it resides and is syntactically similar to a method

NOT a method. Why?? cannot invoke multiple times ... only invoked once in the lifetime of the object

do not have return types and they cannot return values not even void.

called when object is created

can be used to initialise the object immediately upon creation.

JVM first allocates memory for variables and then executes the constructor to initialise instance var

JVM calls automatically when we create obj

defines what happens when obj of cls is created

default: one without parameters

explicit default:

all parametrised constructor are explicitly defined

implicit is default... cant have implicit parametrised

first JVM checks no. of parameters then type of parameters

static keyword: whenever we make any variable as static it belongs to a class. If it is defined in class it belongs to obj but if declared in class using static in class it belongs to **class**

defined in class: instance variable

there should only be one variable and should belong to class like methods. This won't be obj property

whose copy should not be created and should belong to the class then it is called static variable.

main() method is static in nature as JVM can call without creating object.

Without main method JVM will have to create an object but it won't know which

constructor to be called.

Whenever a method is declared as static it can be directly invoked without creating an object. static methods belong to class, so as soon as the class is loaded into JVM the static method would be available to be invoked.

static methods cannot use instance variables as for calling static method you dont need to create object and without object instance variables cant be created.

we can use and declare local, static variables.

static is modifier, can be used with variable, methods, block, class, and import. memory for static members is allocated only once at class loading.

[can reference variables be declared as static?- singleton design pattern](#)

perm gen- the place where static variable occupies its memory

can static variable be declared in main as local variable?

illegal start of expression --- error will be give if you declare that way

so no local variables can NEVER be static

can local variable considered as variables which define state.. can it ever be state variable?

no coz local variable is method variable.

static variable can be used only in static method.

Initializers:

`{}`-- is called **instance initializer**: a block of code without any name.

this is called just before the constructor. This code of block is invoked by the JVM.

new-object creation- look at contrcutor- instance inializer execution- constructor execution

```
{
    System.out.println("Instance initializer called");
}
```

instance intializer can be intiliazed multiple times.they are executed in the order in which they are written

initializers dont take parameters.

suppose a code should be executed and the resultant of that should be given to constrcutor.

static initiazier--

static comes alive when class is loaded

no constructor is called even then it will give output once you load the class

why do we need? in order to initialize static variables or to execute any piece of code when the class is getting loaded(connection to aadhar servers)

constructor chaining-- in a class there can be several constructors present(overloaded constructors) and all these constructors should be invoked while creating a single object, then we use the concept of constructor chaining.

overloading- same name but different parameters

rule: this should always be the first line to the constructor

why do we need?

depending on data available different constructors are invoked resulting in diifferent states of the object, but if all the objects need to come together and have the same state then we need to chain these constructors.

What is inheritance.

when one class(A) attains state state and behaviour from another class(B), it is said to be following inheritance property.

state- variables

behaviour- methods

members- initialisers, constructors : cannot be inherited

static can be inherited

A is the child, derived, specialized.

B becomes the parent, base, generalized.

why do we require inheritance?

code reusability

changes made in base class will affect all derived classes.

we use extends keyword in btwn derived and base class for inheritance

whenever a child class object gets created, a base class object gets created prior to it.

new- space on the heap- name of class- go to class definition- go to the

extends person- go to person- look at person- create instance var- initializer-

constructors- heap take obj nd create copy- go back to teacher- look at class-

take state- take copy- take var- give it to teacher

new- teacher def- extends Person- create copy of instance var- go back to heap - give instance var copy to obj of person - check initializers-execute in order-call constructor of person- after that come back to definition- read teacher take instance var- create copy- give it to teacher- call initializers- constructors

```
/*
```

```
Pracice Program
```

```
*/
```

```
class Teacher extends Person{
    {
        System.out.println("Teacher initializer");
    }
    float exp;
    Teacher(){
        System.out.println("Constructor of person");
    }
    void work() {
        System.out.println("Teaching students");
    }
}
```

```
class Student extends Person{
    String name;
    int age;
    float cgpa;
    void study() {
        System.out.println("Studying lessons");
    }
}
```

```
class Staff extends Person{
    String name;
    int age;
    void maintain() {
        System.out.println("Maintaining school facilities");
    }
}
```

```
class Person{
    {
        System.out.println("Person initializer");
    }
}
```

```

    }
    int age;
    String name;
    Person(){
        System.out.println("Constructor of tr");
    }
    void eat(){
        System.out.println("Eat called");
    }
    void display(){
        System.out.println("Name: "+name);
        System.out.println("Age: "+age);
    }
}

```

```

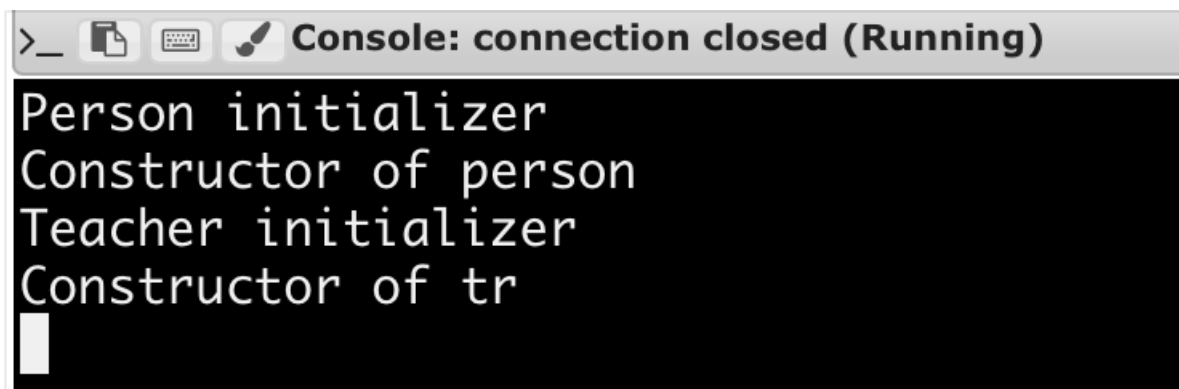
public class test1
{
    public static void main(String[] args)
    {
        Teacher t=new Teacher();

    }

}

```

o/p:



The screenshot shows a console window titled "Console: connection closed (Running)". The output text is as follows:

```

Person initializer
Constructor of person
Teacher initializer
Constructor of tr

```

IT CAN INHERIT static variables.

super(): it is like this for methods
create a librarian and a principal

Overriding:

when a child class/derived class requires the same functionality from the parent/base class but wants to change its implementation, then it needs to re-define that method in its own class. This process is called as overriding.

there are several rules for overriding:

- the name of the method should be same
- parameter list(number of parameters, datatype of parameters, order of parameters) should be same
- return type should be same or should be the subclass of the return type of the overridden method(law of co-variance)
- access modifiers should not be restricted
- parent class- overridden method
- subclass - overriding method
- final methods cannot be overridden

when return types doesn't match then there is no overriding

if parameters are different then only methods will be overloaded

```
/*
```

```
Pracice Program
```

```
*/
```

```
class Teacher extends Person{
    int exp;
    Teacher(String name,int age,int exp){
        super(name,age);
        this.exp=exp;
    }
    void work() {
        System.out.println("Teaching students");
    }
}
```

```
class Student extends Person{

    float cgpa;
    Student(){
        super("name",10);
    }
    Student(String name,int age){
        super(name,age);
    }
}
```



```

    }
    Student(String name,int age,float cgpa){
        super(name,age);
        this.cgpa=cgpa;
    }

    void study() {
        System.out.println("Studying lessons");
    }
}

class Staff extends Person{
    float sal;
    void maintain() {
        System.out.println("Maintaining school facilities");
    }
    Staff(String name,int age,float sal){
        super(name,age);
        this.sal=sal;
    }
}

class Person
{
    String name; int age;
    Person(String name, int age)
    {
        this.name = name;
        this.age = age ;
        System.out.println("Default Constructor Called");
    }

    void display()
    {
        System.out.println(name+ " is "+age+" years old.");
    }
    int getDiscount()
    {
        if(age>60)
            return 50;
        else
            return 0;
    }
}

```

```

}class Test
{
    public static void main (String[] args) {
        Teacher t = new Teacher("Sujata",34,8);
        Teacher t1 = new Teacher("Meera",24,3);
        Teacher t2 = new Teacher("Priya",75,4);
        Student s = new Student("Shashi",19);
        Student s1 = new Student("Kiran",20,9.0f);
        Student s2 = new Student();//default students should have name as John
        Doe with minimum age of 17
        Staff ss = new Staff("Rishi",45,78000.0f);
        t.display();
        t1.display();
        t2.display();
        s.display();
        s1.display();
        s2.display();
        ss.display();
        System.out.println(t2.getDiscount());

    }
}

```

final keyword: we cannot change the data in this variable

all final variables are declared as static...why?

a final instance variable cannot change its value so there is no point creating multiple copies of it

whenever a method is made final you can create derived methods, but cannot be overridden

constructors and initializer cant be final

class can be declared as final

you cannot inherit the class declared as final

abstract keyword:

abstract methods cannot have a body nor a variable

whenever a method is declared as abstract it means it should not have a body

why is it necessary for a sub class to override an abstract method in its own class?

like old man came is he inheriting name, age, sleep, discount are methods there

and drink is declared as abstract

i have a base class and have some method in it: sleep

whenever a base class has got a particular method and that particular method is getting overridden by all the classes, is anybody going to use default implementation? no everyone gonna use their implementation of the behaviour of the method. then as a developer i will delete the body of default implementation and declare it as abstract

the base class wishes a behaviour should be added, but base class is unaware how subclass is gonna use the behaviour and doesn't wanna give default implementation, i want every class to have drink, behaviour should be same but implementation you do on your own.

suppose i have a class called as 'car'

there is a particular method called accelerate, break in car

now diesel, petrol and electric are subclasses

accn in diesel is different, than electric than petrol

now accn will become abstract in 'car' class

if car doesn't have accn and electric car sub class is there, then electric car will k it should accn? no

now another car will come, like cng

suppose base car is saying should have steering wheel, any class extended from car should have a wheel, wheel can be anything: 360/180 degree

whenever a method is declared as abstract in a class, class should be declared as abstract too

abs class= all methods abstract, implemented + abs, all methods implemented

methods can never be abstract and final simul as final demands no overriding but abstract demands overriding

class can never be abstract and final simul as final demands no inheritance but abstract demands inheritance

polymorphism: